

Preprocesamiento de Datos

Clase 3 - Inteligencia Artificial

Dr. Diego Stalder

Universidad Nacional de Asunción
Facultad de Ingeniería
Ingeniería Mecatrónica

29 de Julio de 2026

Auxiliar: Ing. Enrique Paiva

Contenido de la Clase

- 1 Introducción al Preprocesamiento
- 2 NumPy y Pandas
- 3 Normalización y Estandarización
- 4 Encoding de Variables Categóricas
- 5 Manejo de Datos Faltantes
- 6 Preprocesamiento en Scikit-learn
- 7 Dataset Ejemplo

Principio Fundamental

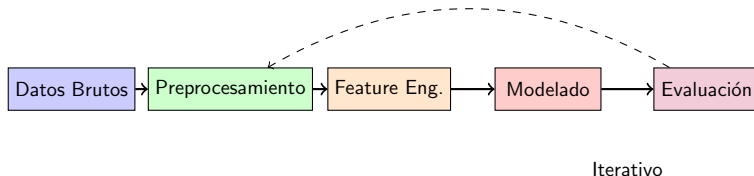
Garbage In, Garbage Out

- Los modelos de ML son tan buenos como los datos que reciben
- **80 %** del tiempo en un proyecto de ML se dedica al preprocesamiento
- Un buen preprocesamiento puede mejorar el rendimiento del modelo en un **30-50 %**

Consecuencias de Datos Pobres

- Malas predicciones / modelado
- Sesgos en los resultados
- Sobreajuste al ruido
- Dificultad en la interpretación

El Pipeline de Machine Learning



El Preprocesamiento es Clave

- Es **obligatorio** para algunos modelos (SVM, Redes Neuronales)
- Disminuye sesgos
- Identifica datos redundantes e irrelevantes
- Reduce la dimensionalidad
- Mejora la convergencia

Estructuras Principales

- **Array**: Estructura multidimensional homogénea
- **Vectorización**: Operaciones sin bucles explícitos
- **Broadcasting**: Operaciones con arrays de diferentes dimensiones

Operación	Ejemplo	Resultado
Creación	<code>np.array([1,2,3])</code>	Array de 3 elementos
Indexación	<code>arr[0:2]</code>	Primeros 2 elementos
Estadística	<code>np.mean(arr)</code>	Media del array
Álgebra lineal	<code>np.dot(A, B)</code>	Producto punto

Ventajas sobre listas Python

- Mucho **más rápido** (C en el backend)
- **Menor consumo** de memoria
- **Operaciones vectorizadas** optimizadas

Estructuras Principales

- **Series:** Array unidimensional etiquetado
- **DataFrame:** Tabla bidimensional con columnas etiquetadas

Método	Función	Ejemplo
head()	Primeras filas	<code>df.head(10)</code>
info()	Información del DF	<code>df.info()</code>
describe()	Estadísticas descriptivas	<code>df.describe()</code>
isnull()	Valores faltantes	<code>df.isnull().sum()</code>
loc/iloc	Selección	<code>df.loc[df['col'] > 5]</code>
groupby()	Agrupación	<code>df.groupby('col').mean</code>

Importante

`pd.read_csv()` es el método más común para cargar datos tabulares.

¿Por qué Escalar los Datos?

Problema: Escalas Diferentes

- Edad: 18-65 años
- Ingreso: 10,000-200,000
- Distancia: 1-100 km

Los modelos basados en distancia (SVM, KNN) se ven afectados.

Modelos Afectados

- Regresión Lineal
- SVM
- KNN
- Redes Neuronales
- PCA
- K-Means

Algoritmos NO Afectados

Árboles de Decisión, Random Forest, Gradient Boosting.

Normalización (Min-Max Scaling)

Fórmula

$$X_{\text{normalizado}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (1)$$

- Escala los datos al rango **[0, 1]**
- Mantiene la forma de la distribución original
- **Sensible a outliers**
- Usar cuando los datos tienen límites conocidos

Original	Min-Max	Z-score
10	0.00	-1.41
20	0.50	-0.71
30	1.00	0.00
40	1.50	0.71
50	2.00	1.41

En Scikit-learn

```
from sklearn.preprocessing import MinMaxScaler
```

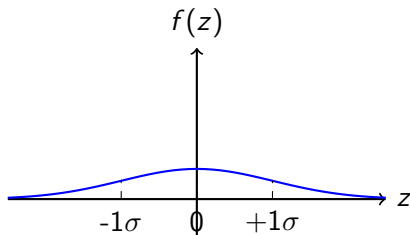

Estandarización (Z-Score)

Fórmula

$$x_{\text{estandarizado}} = \frac{x - \mu}{\sigma} \quad (2)$$

Donde μ es la media y σ la desviación estándar.

- Media = **0**, Desviación = **1**
- **Recomendado** para SVM, PCA, Redes Neuronales
- **Más robusto** a outliers que Min-Max
- Asume distribución aproximadamente normal



En Scikit-learn

```
from sklearn.preprocessing import StandardScaler
```

Comparación: Normalización vs Estandarización

Criterio	Min-Max	Z-Score
Rango	[0, 1]	Sin límites
Media	Variable	0
Desviación	Variable	1
Outliers	Muy sensible	Robusto
Distribución	Mantiene forma	Asume normal
Interpretabilidad	Alta	Media
Uso típico	Redes Neuronales	SVM, PCA

Cuándo usar cada una

- **Min-Max:** Cuando los datos deben estar en un rango fijo (ej. imágenes [0,1])
- **Z-Score:** Cuando el modelo asume normalidad o usa distancias

Variables Categóricas

Tipos de Variables Categóricas

- **Nominales:** Sin orden intrínseco (ej. color, ciudad)
- **Ordinales:** Con orden intrínseco (ej. educación, tamaño)
- **Cíclicas:** Con periodicidad (ej. día de la semana, mes)

Problema

Los modelos de ML no entienden texto. **Necesitamos convertir a números.**

Tipo	Ejemplo	Método
Nominal	Color: rojo, azul, verde	One-Hot Encoding
Ordinal	Tamaño: S, M, L	Label Encoding
Cíclica	Mes: 1, 2, ..., 12	Seno/Coseno

Label Encoding

Definición

Asignar un número entero a cada categoría.

Categoría	Código
Rojo	0
Azul	1
Verde	2

Problema

Introduce un orden falso.

Modelo podría pensar: verde > azul
> rojo.

¿Cuándo usarlo?

Solo para variables **ordinales** donde el orden es significativo.

Ejemplos: nivel de educación (primaria, secundaria, universidad).

En Scikit-learn

```
from sklearn.preprocessing import LabelEncoder
```

One-Hot Encoding

Definición

Crear una variable binaria por cada categoría.

Color	Rojo	Azul	Verde
Rojo	1	0	0
Azul	0	1	0
Verde	0	0	1

Ventajas

- **No introduce orden falso**
- Fácil de interpretar
- Compatible con la mayoría de modelos

Desventajas

- **Maldición dimensional** con muchas categorías
- Correlación entre variables dummy
- Problema de la "trampa de la dummy"

One-Hot Encoding - Consideraciones

Trampa de la Dummy (Dummy Trap)

Si tenemos k categorías y creamos k variables dummy, hay **multicolinealidad perfecta**.

Solución: Crear $k - 1$ variables dummy o usar `drop='first'`.

Manejo de Categorías Raras

- Agrupar categorías poco frecuentes en ".Otros"
- Usar `min_frequency` o `max_categories` en `OneHotEncoder`

Alternativas

- **Target Encoding:** Reemplazar categoría por promedio del target
- **Frequency Encoding:** Reemplazar por frecuencia
- **Embedding:** Para variables con muchas categorías

Problema

Variables cíclicas (día, mes, hora) **no son lineales**.

Ejemplo: Enero (1) y Diciembre (12) están cerca, pero numéricamente lejos.

Solución: Transformación Seno/Coseno

$$x_{\sin} = \sin\left(\frac{2\pi x}{T}\right) \quad (3)$$

$$x_{\cos} = \cos\left(\frac{2\pi x}{T}\right) \quad (4)$$

Donde T es el periodo (ej. 12 para meses, 7 para días).

Ventajas

- Mantiene la **relación cíclica**
- Punto (sin, cos) forma un círculo unitario
- Distancia euclidiana refleja similitud cíclica

Binning y Discretización

Definición

Convertir variables continuas en variables categóricas.

Equal Width Binning

Dividir en intervalos de **igual ancho**.

Ejemplo: Edad $\rightarrow$ [0-18], [19-35], [36-60], [60+]

Equal Frequency Binning

Dividir en intervalos con **igual número de observaciones**.

Cada bin contiene aproximadamente el mismo número de datos.

Consideraciones

- **Pierde información** (se discretiza)
- **Outliers** pueden distorsionar los bins
- Elegir el número de bins es crítico

En Scikit-learn

```
from sklearn.preprocessing import KBinsDiscretizer
```

Clasificación de Datos Faltantes

- **MCAR** (Missing Completely at Random)
 - La falta es completamente aleatoria
 - Ejemplo: Error de medición aleatorio
- **MAR** (Missing at Random)
 - La falta depende de otras variables observadas
 - Ejemplo: Personas con altos ingresos no reportan salario
- **MNAR** (Missing Not at Random)
 - La falta depende del valor faltante
 - Ejemplo: Personas con depresión no responden encuestas

Estrategias para Datos Faltantes

Eliminación

- **Listwise Deletion:** Eliminar filas con NA
- **Pairwise Deletion:** Eliminar solo para análisis específicos
- **Ventaja:** Simple y rápido
- **Desventaja:** Pierde información valiosa

Imputación

- **Media/Mediana/Moda**
- **Imputación por regresión**
- **KNN Imputation**
- **MICE** (Multiple Imputation)
- **Ventaja:** Retiene datos
- **Desventaja:** Puede introducir sesgo

Métodos de Imputación

Método	Descripción	Scikit-learn
Media	Rellenar con la media de la columna	<code>SimpleImputer(strategy="mean")</code>
Mediana	Rellenar con la mediana	<code>SimpleImputer(strategy="median")</code>
Moda	Rellenar con el valor más frecuente	<code>SimpleImputer(strategy="most_frequent")</code>
Constante	Rellenar con un valor fijo	<code>SimpleImputer(strategy="constant", fill_value=0)</code>
KNN	Basado en vecinos cercanos	<code>KNNImputer</code>
MICE	Modelo predictivo iterativo	<code>IterativeImputer</code>

Recomendación

- **Menos del 5 %:** Eliminar o imputar simple
- **5 % - 20 %:** Imputación con KNN o MICE
- **¿ 20 %:** Considerar si la variable es útil

ColumnTransformer - Pipelines Múltiples

Problema

Diferentes columnas necesitan diferente preprocesamiento.

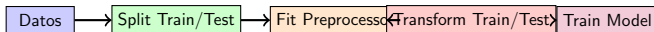
Solución: ColumnTransformer

```
from sklearn.compose import ColumnTransformer
```

```
preprocessor = ColumnTransformer([  
    ('num', StandardScaler(), numeric_cols),  
    ('cat', OneHotEncoder(), categorical_cols),  
    ('impute', SimpleImputer(), missing_cols)  
])
```

Componente	Aplicación	Ejemplo
Pipeline	Secuencia de pasos	Escalar → Modelo
ColumnTransformer	Diferentes columnas	Númericas → Escalar Categorías → One-Hot
FunctionTransformer	Transformación personalizada	Log transform

Pipeline Completo de Preprocesamiento



Regla de Oro

¡Nunca ajustes el preprocesador en el conjunto de test!

Siempre: `fit_transform(train)` → `transform(test)`

Mejor Práctica

Usar `sklearn.pipeline.Pipeline` para asegurar que el preprocesamiento y el modelo se aplican correctamente.

Pasos para NLP

- 1 **Tokenización:** Dividir en palabras
- 2 **Stopwords:** Eliminar palabras comunes
- 3 **Stemming/Lematización:** Reducir a raíz
- 4 **Vectorización:** Convertir a números

Métodos de Vectorización

- **CountVectorizer:** Frecuencia de palabras
- **TF-IDF:** Term Frequency - Inverse Document Frequency
- **Word Embeddings:** Word2Vec, GloVe

Fórmula TF-IDF

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{N}{\text{DF}(t)} \right) \quad (5)$$

Donde:

- $\text{TF}(t, d)$: Frecuencia del término t en el documento d
- $\text{DF}(t)$: Número de documentos que contienen t
- N : Número total de documentos

Resumen de Preprocesamiento

Técnica	Aplicación	Scikit-learn
Estandarización	Variables numéricas	StandardScaler
Normalización	Variables numéricas	MinMaxScaler
One-Hot Encoding	Categóricas nominales	OneHotEncoder
Label Encoding	Categóricas ordinales	LabelEncoder
Codificación Cíclica	Variables temporales	FunctionTransformer
Imputación	Datos faltantes	SimpleImputer
Binning	Discretización	KBinsDiscretizer
TF-IDF	Texto	TfidfVectorizer

Dataset: Ames Housing

Contexto

Dataset de ventas de casas en Ames, Iowa (EEUU).

- **Observaciones:** 1,460
- **Variables:** 81
- **Objetivo:** Predecir precio de venta (SalePrice)

Tipo de Variable	Ejemplos	Preprocesamiento
Numéricas	LotArea, GrLivArea	StandardScaler
Catóricas	Neighborhood, HouseStyle	OneHotEncoder
Cíclicas	MoSold (mes)	Seno/Coseno
Faltantes	LotFrontage	SimpleImputer

Impacto del Preprocesamiento

Sin preprocesamiento: RMSE $\approx$ 35,000

Con preprocesamiento: RMSE $\approx$ 26,000

Mejora del 25 %

?

¿Preguntas?

Dr. Diego Stalder
dstalder@ing.una.py